



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Piano di Qualifica

Versione	1.0.0
Stato	Approvato
Redazione	Elisa Marchioro, Giulia Barzon
Verifica	Tutto il gruppo
Approvazione	Joseph Grant
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica
1.0.0	26/02/2026	Joseph Grant	Approvazione finale del documento	
0.2.0	24/02/2026	Giulia Barzon	Aggiornamento cruscotto di qualità	Elisa Marchioro e Chiara Grossele
0.1.5	21/02/2026	Giulia Barzon	Aggiunti test analisi dinamica	Elisa Marchioro e Chiara Grossele
0.1.4	05/02/2026	Giulia Barzon	Aggiornamento cruscotto di qualità	Elisa Marchioro e Chiara Grossele
0.1.3	11/01/2026	Elisa Marchioro	Aggiunte metriche qualità di prodotto	Giulia Barzon
0.1.2	11/01/2026	Giulia Barzon	Aggiunte metriche qualità di processo	Elisa Marchioro
0.1.1	27/12/2025	Joseph Grant	Aggiunta sezione 'verifica' al versionamento	Giulia Barzon
0.1.0	15/12/2025	Giulia Barzon	Creazione del documento, scrittura introduzione e checklist	Elisa Marchioro

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
1.1	Scopo del documento	1
1.2	Scopo del prodotto	1
1.3	Glossario	1
1.4	Riferimenti	1
1.4.1	Riferimenti Normativi	1
1.4.2	Riferimenti Informativi	2
2	Metriche di Qualità	3
2.1	Qualità di Processo	3
2.1.1	Processi Primari di Fornitura _G	3
2.1.2	Processi Primari di Sviluppo _G	6
2.1.3	Processi di Supporto _G : Documentazione e Qualità	7
2.2	Qualità di Prodotto	8
2.2.1	Adeguatezza Funzionale _G	8
2.2.2	Qualità dell'Output Generativo _G	8
2.2.3	Affidabilità _G	8
2.2.4	Efficienza _G	9
2.2.5	Usabilità _G	9
2.2.6	Manutenibilità _G	9
3	Strategia di Verifica (Testing)	11
3.1	Analisi Statica _G	11
3.1.1	Checklist per la Verifica (Analisi Statica tramite Inspection _G)	11
3.2	Analisi Dinamica _G	13
3.2.1	Test di Unità _G	13
3.2.2	Test di Integrazione _G	13
3.2.3	Test di Sistema _G	14
4	Cruscotto di Valutazione della Qualità_G	15
4.1	Qualità di Processo	15
4.1.1	Estimated at Completion (EAC)	15
4.1.2	Planned Value (PV) e Earned Value (EV)	16
4.1.3	Schedule Performance Index (SPI)	17
4.1.4	Cost Performance Index (CPI)	18
4.1.5	Cost Variance (CV)	19
4.1.6	Requirements Stability Index (RSI)	20
4.1.7	Rischi Non Calcolati (RNC)	21
4.1.8	Metriche Soddisfatte (MS)	22

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di definire la strategia, le metodologie e le risorse pianificate per garantire la qualità del prodotto software **"Second Brain"** e l'efficienza dei processi di sviluppo adottati dal gruppo. Esso funge da riferimento principale per le attività di **verifica** (accertamento che il prodotto sia costruito correttamente) e **validazione** (accertamento che il prodotto giusto sia stato costruito), stabilendo gli standard qualitativi che ogni artefatto deve soddisfare prima di essere approvato. In questo contesto, il termine **"Qualità"** viene inteso secondo due prospettive complementari:

- **Qualità di Processo:** L'aderenza del team al metodo di lavoro definito nelle *Norme di Progetto*, basato sul miglioramento continuo e sull'efficienza delle procedure di sviluppo e test.
- **Qualità di Prodotto:** La capacità del software di soddisfare i requisiti funzionali e non funzionali esplicitati nell'*Analisi dei Requisiti*.

Il documento è destinato al committente Prof. Vardanega e Prof. Cardin, all'azienda proponente Zucchetti S.p.A. e al gruppo di sviluppo ByteMe.

1.2 Scopo del prodotto

Il progetto **Second Brain**, proposto dall'azienda Zucchetti, mira a sviluppare un editor di testo avanzato basato sul linguaggio Markdown e potenziato dall'integrazione di Large Language Models (LLM). L'applicazione web consentirà all'utente di:

- Scrivere e visualizzare in tempo reale testi formattati in Markdown;
- Interagire con un LLM per generare, riassumere, riscrivere, tradurre e analizzare testi;
- Sfruttare il metodo dei "Sei cappelli per pensare" di Edward De Bono per ottenere analisi critiche multi-prospettiche;
- Fornire prompt testuali per delegare all'AI la stesura completa del contenuto (*Distant Writing*).

L'obiettivo è creare uno strumento di supporto alla scrittura creativa e al note-taking avanzato, esplorando le potenzialità pratiche degli LLM nel flusso di lavoro quotidiano.

1.3 Glossario

Per evitare ambiguità e garantire un linguaggio comune all'interno del gruppo e con i committenti, i termini tecnici utilizzati in questo documento sono definiti in modo approfondito nel documento *Glossario*, situato nella repository del gruppo alla sezione *Documenti interni*. Per facilitare la lettura, i termini che presentano una definizione specifica nel glossario sono contrassegnati con una G a pedice.

1.4 Riferimenti

1.4.1 Riferimenti Normativi

- **Capitolato d'appalto C6:** Second Brain - *Zucchetti*
- **Norme di Progetto:** Documento interno del gruppo che definisce regole, strumenti e procedure.

- **Regolamento del progetto didattico:** Corso di Ingegneria del Software, Università degli Studi di Padova.

1.4.2 Riferimenti Informativi

- **ISO/IEC 12207:** *Systems and software engineering — Software life cycle processes*. (Standard di riferimento per il ciclo di vita del software).
- **ISO/IEC 25010:** *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*. (Modello di riferimento per la qualità del prodotto software).
- **Slide del corso di Ingegneria del Software:**
 - T9 - Verifica e validazione.
 - T10 - Analisi statica.
 - T11 - Analisi dinamica.
- **Slide e dispense del corso opzionale:** *Metodi e Tecnologie per lo Sviluppo Software*.
- **Documentazione tecnica strumenti:** *GitHub Actions Documentation*

2 Metriche di Qualità

2.1 Qualità di Processo

2.1.1 Processi Primari di Fornitura_G

In questa sezione vengono definite le metriche relative alla gestione manageriale ed economica del progetto. L'obiettivo è monitorare l'allocazione delle risorse, il rispetto delle scadenze temporali e l'aderenza al preventivo definito in fase di candidatura. Per garantire un controllo oggettivo sull'avanzamento dei lavori, il gruppo adotta lo standard **EVA (Earned Value Analysis)_G**, che permette di integrare la misurazione di tempi, costi e valore prodotto in un unico framework quantitativo.

Budget at Completion_G (BAC)

Questo valore indica il budget totale massimo a disposizione del gruppo per lo svolgimento del progetto; rappresenta il limite di spesa totale.

Il preventivo iniziale, calcolato in fase di candidatura, ammonta a **11.395 euro** per un totale di 558 ore di lavoro.

- **Soglia Accettabile:** minimizzato.
- **Soglia Ottimale:** minimizzato.

Planned Value_G (PV)

Valore monetario (o in ore) delle attività pianificate che avrebbero dovuto essere completate entro la data corrente. Si distingue in:

- **Sprint Planned Value (SPV)**
Valore pianificato per un singolo Sprint. Calcolato come somma delle ore ruolo (OR) preventivate moltiplicate per il costo orario (CR).

Calcolo	Soglia Accettabile	Soglia Ottimale
$SPV = \sum (OR \times CR)$	> 0	> 0

Tabella 2: Parametro Sprint Planned Value (SPV)

- **Project Planned Value (PPV)**
Valore pianificato cumulativo per l'intero progetto (somma degli SPV passati).

Calcolo	Soglia Accettabile	Soglia Ottimale
$PPV = \sum SPV_{passati}$	> 0	> 0 $\leq BAC$

Tabella 3: Parametro Project Planned Value (PPV)

Actual Cost_G (AC)

Costo effettivo sostenuto (ore lavorate $\times$ costo orario) per il lavoro svolto finora. Si distingue in:

- **Sprint Actual Cost (SAC)**
Costo sostenuto nel singolo Sprint.

Calcolo	Soglia Accettabile	Soglia Ottimale
$SAC = \sum \text{costi sostenuti nello Sprint}$	$\leq SPV + 10\%$	$\leq SPV$

Tabella 4: Parametro Sprint Actual Cost (SAC)

- **Project Actual Cost (PAC)**

Costo totale sostenuto dall'inizio del progetto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$PAC = \sum_{s \in S} SAC_s$	$\leq BAC$	$\leq BAC$

Tabella 5: Parametro Project Actual Cost (PAC)

Earned Value_G (EV)

Rappresenta il valore del lavoro effettivamente completato e verificato alla data corrente. Per calcolarlo si adotta la tecnica di misurazione fisica **0/100_G**: il valore pianificato di un'attività viene accreditato solo se l'attività è completata al 100%.

- **Sprint Earned Value (SEV)**

Valore guadagnato nel singolo Sprint. È la somma del Planned Value delle sole attività completate secondo la Definition of Done.

Calcolo	Soglia Accettabile	Soglia Ottimale
$SEV = \sum PV_{completati}$	$\geq 90\%$ del SPV	$= SPV$

Tabella 6: Parametro Sprint Earned Value (SEV)

- **Project Earned Value (PEV)**

Valore guadagnato cumulativo dall'inizio del progetto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$PEV = \sum SEV_{passati}$	$\geq 90\%$ del PPV	$= PPV$

Tabella 7: Parametro Project Earned Value (PEV)

Schedule Variance_G (SV)

Misura lo scostamento temporale del progetto rispetto alla pianificazione iniziale. Indica se il progetto è in anticipo, in pari o in ritardo in termini di valore prodotto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$SV = EV - PV$	$SV \geq -10\%$ del PV	$SV \geq 0$

Tabella 8: Metrica Schedule Variance (SV)

Cost Variance_G (CV)

Misura l'efficienza economica del progetto, confrontando il valore prodotto (guadagnato) con i costi effettivamente sostenuti. Indica se stiamo spendendo più del necessario per il lavoro svolto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$CV = EV - AC$	$CV \geq -8\% \text{ del EV}$	$CV \geq 0$

Tabella 9: Metrica Cost Variance (CV)

Cost Performance Index_G (CPI)

Indice di efficienza dei costi. Rapporto tra il valore guadagnato e il costo effettivo sostenuto. Un valore inferiore a 1 indica che il costo sostenuto è superiore al valore prodotto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$CPI = \frac{PEV}{PAC}$	$CPI \geq 0.95$	$CPI \geq 1.00$

Tabella 10: Metrica Cost Performance Index (CPI)

Schedule Performance Index_G (SPI)

Indice di efficienza temporale; è il rapporto tra il valore guadagnato e il valore pianificato. Indica la velocità di avanzamento rispetto al piano.

Calcolo	Soglia Accettabile	Soglia Ottimale
$SPI = \frac{EV}{PV}$	$SPI \geq 0.90$	$SPI \geq 1.00$

Tabella 11: Metrica Schedule Performance Index (SPI)

Estimated at Completion_G (EAC)

Stima aggiornata del costo totale finale del progetto, calcolata proiettando l'efficienza attuale (CPI) fino alla fine del progetto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$EAC = \frac{BAC}{CPI}$	$EAC \leq BAC$	$EAC \leq BAC$

Tabella 12: Metrica Estimated at Completion (EAC)

2.1.2 Processi Primari di Sviluppo_G

Questa sezione raggruppa le metriche utilizzate per valutare la qualità delle attività di realizzazione tecnica del prodotto. Il monitoraggio si focalizza sulla **stabilità dei requisiti**, per controllare l'evoluzione dello scopo del progetto ed evitare cambiamenti incontrollati, e sulla **qualità strutturale del codice**, analizzando la complessità e la dimensione del software prodotto per garantirne la manutenibilità futura.

Requirements Stability Index_G (RSI)

Misura la stabilità dei requisiti nel corso del tempo, indicando la percentuale di requisiti rimasti invariati rispetto al totale. Un valore basso indica frequenti cambiamenti nello scopo del progetto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$RSI = (1 - \frac{N_{modifiche}}{N_{totali}}) \times 100$	$RSI \geq 80\%$	$RSI = 100\%$

Tabella 13: Metrica Requirements Stability Index (RSI)

Copertura dei Requisiti_G (CR)

Percentuale di requisiti implementati e verificati (soddisfatti) rispetto al totale pianificato, suddivisi per importanza (Obbligatori, Desiderabili, Opzionali).

Calcolo	Soglia Accettabile	Soglia Ottimale
$C = \frac{N_{soddisfatti}}{N_{totali}} \times 100$	Obbl: 100% Desid: $\geq 40\%$ Opz: $\geq 0\%$	Obbl: 100% Desid: 100% Opz: 100%

Tabella 14: Metrica Copertura dei Requisiti (CR)

Lines of Code_G (LOC)

Misura la dimensione fisica del software tramite il conteggio delle righe di codice sorgente (esclusi commenti e righe vuote). Questa è una metrica informativa di monitoraggio, quindi non vengono valutate le soglie.

Calcolo	Soglia Accettabile	Soglia Ottimale
Somma righe logiche nei file sorgente	N/A	N/A

Tabella 15: Metrica Lines of Code (LOC)

Complessità Ciclomatica_G (McCabe)

Misura la complessità del flusso di controllo di un metodo o funzione calcolando il numero di cammini linearmente indipendenti. Una complessità elevata indica codice difficile da testare e mantenere.

Calcolo	Soglia Accettabile	Soglia Ottimale
$v(G) = E - N + 2P$	≤ 8	≤ 6

Tabella 16: Metrica Complessità Ciclomatica (McCabe)

2.1.3 Processi di Supporto_G: Documentazione e Qualità

Le metriche esposte in questa sezione valutano l'efficacia delle attività trasversali che supportano il ciclo di vita del software. L'attenzione è posta sulla **qualità della documentazione**, assicurando che sia leggibile, corretta e priva di ambiguità, e sull'efficacia del **processo di verifica**, misurando la capacità del gruppo di identificare e risolvere rischi e difetti in modo proattivo.

Indice di Gulpease_G (IG)

Valuta la leggibilità di un testo redatto in lingua italiana per garantire che la documentazione sia comprensibile ai membri del gruppo e agli stakeholder esterni.

Calcolo	Soglia Accettabile	Soglia Ottimale
$IG = 89 + \frac{300 \times N_{frasi} - 10 \times N_{lettere}}{N_{parole}}$	$G \geq 40$	$G \geq 60$

Tabella 17: Metrica Indice di Gulpease (IG)

Correttezza Ortografica (CO)

Misura il numero di errori ortografici o grammaticali residui nei documenti individuati durante i processi di verifica o revisione.

Calcolo	Soglia Accettabile	Soglia Ottimale
Conteggio errori manuale/tool	0	0

Tabella 18: Metrica Correttezza Ortografica (CO)

Rischi Non Calcolati (RNC)

Quantità di rischi che si verificano durante il progetto ma che non erano stati previsti o analizzati in fase di candidatura e nel documento Piano di Progetto.

Calcolo	Soglia Accettabile	Soglia Ottimale
Conteggio rischi imprevisti	≤ 2	0

Tabella 19: Metrica Rischi Non Calcolati (RNC)

Metriche Soddisfatte (MS)

Percentuale di metriche (tra tutte quelle definite nel presente documento) che rientrano nelle soglie di accettabilità ad ogni verifica di periodo. Indica lo stato di salute generale del progetto.

Calcolo	Soglia Accettabile	Soglia Ottimale
$MS = \frac{metricheOK}{TOTmetriche} \times 100$	$\geq 75\%$	100%

Tabella 20: Metrica Metriche Soddisfatte (MS)

2.2 Qualità di Prodotto

La qualità di prodotto nel software indica quanto bene un sistema riesce a soddisfare i bisogni degli utenti e gli obiettivi per cui è stato creato. Non significa solo che il programma funzioni, ma anche che sia affidabile, veloce, facile da usare e capace di essere migliorato nel tempo. Un software di buona qualità è quindi utile, stabile e adatto a evolversi insieme alle esigenze degli utenti.

2.2.1 Adeguatezza Funzionale_G

Indica quanto il software realizza correttamente tutte le funzioni richieste per soddisfare gli obiettivi dell'utente.

Metrica	Scopo della metrica	Soglia Accettabile	Soglia Ottimale
Requirement Coverage	Verifica che tutte le funzionalità principali dell'editor siano implementate	90%	98%
Acceptance Test Pass Rate	Controlla che le funzionalità principali funzionino correttamente	95%	99%
Functional Defect Density	Misura quanti bug ci sono per funzione principale	0.5 bug/funzione	0.1 bug/funzione

Tabella 21: Soglie metriche funzionalità del prodotto

2.2.2 Qualità dell'Output Generativo_G

Misura quanto le risposte prodotte dal sistema sono corrette, coerenti e utili per l'utente.

Metrica	Scopo della metrica	Soglia Accettabile	Soglia Ottimale
Accuracy	Misura quanto i suggerimenti o correzioni AI sono corretti	85%	95%
Hallucination Rate	Misura quanto l'AI suggerisce contenuti errati o inventati	10%	2%
Human Rating	Valutazione soggettiva della qualità dei suggerimenti AI da parte degli utenti	3.5 / 5	4.5 / 5
Completion Success Rate	Percentuale di suggerimenti AI accettati dall'utente	75%	90%

Tabella 22: Soglie metriche qualità output AI

2.2.3 Affidabilità_G

Rappresenta la capacità del software di funzionare in modo stabile e continuo senza guasti frequenti.

Metrica	Scopo della metrica	Soglia Accettabile	Soglia Ottimale
Availability	Percentuale di tempo in cui il software è operativo e stabile	99%	99.9%
Error Rate	Percentuale di operazioni che falliscono (salvataggio, apertura file, AI)	1%	0.1%
MTTR	Tempo medio per ripristinare il software dopo un crash	60 minuti	10 minuti

Tabella 23: Soglie metriche affidabilità del prodotto

2.2.4 Efficienza_G

Indica quanto il sistema è veloce e quanto bene utilizza le risorse disponibili.

Metrica	Scopo della metrica	Soglia Accettabile	Soglia Ottimale
Response Time (Latenza Interfaccia)	Tempo medio che l'interfaccia impiega per aggiornarsi correttamente	500 ms	100 ms
AI Processing Time	Tempo medio necessario all'AI per generare un testo	10 sec	3 sec

Tabella 24: Soglie metriche efficienza del prodotto

2.2.5 Usabilità_G

Descrive quanto il software è facile da imparare e da usare per gli utenti.

Metrica	Scopo della metrica	Soglia Accettabile	Soglia Ottimale
Task Success Rate	Percentuale di utenti che completano correttamente un compito (scrittura, salvataggio, uso AI)	80%	95%
SUS Score	Valutazione dell'usabilità tramite questionario standard	68	85
User Satisfaction Rate	Percentuale di utenti soddisfatti delle funzioni AI	75%	90%

Tabella 25: Soglie metriche usabilità del prodotto

2.2.6 Manutenibilità_G

Indica quanto è semplice correggere, modificare ed evolvere il software nel tempo.

Metrica	Scopo della metrica	Soglia Accettabile	Soglia Ottimale
Test Coverage	Percentuale di codice coperto dai test (funzioni base + AI)	70%	90%
Change Failure Rate	Percentuale di modifiche che introducono problemi o bug	15%	5%

Tabella 26: Soglie metriche manutenibilità del prodotto

3 Strategia di Verifica (Testing)

3.1 Analisi Statica_G

3.1.1 Checklist per la Verifica (Analisi Statica tramite Inspection_G)

Questa sezione contiene le liste di controllo che i Verificatori devono utilizzare obbligatoriamente prima di approvare un documento o un pezzo di codice. L'obiettivo è standardizzare il controllo qualità ed evitare errori ricorrenti. Questa checklist_G viene aggiornata progressivamente dai verificatori e permette di evitare gli errori più comuni e semplici da individuare.

Oggetto	Errore	Azione Correttiva
Immagini e Tabelle	Mancanza di didascalia (caption)	Ogni tabella o immagine deve avere una didascalia esplicativa numerata.
Struttura	Sezioni vuote o orfane	Rimuovere o riempire le sezioni che contengono solo il titolo senza testo.
Glossario	Termine non marcato o errato	Verificare che i termini del glossario siano segnati con la "G" a pedice e rispettino il case sensitivity.
Ortografia	Errori di battitura o sintassi	Utilizzare il correttore automatico, verificare personalmente i testi, rimuovere refusi e doppi spazi.
Percorsi File	Path immagini errato	Verificare che i percorsi siano relativi e corretti (es: <code>../UCimg/diagramma.jpg</code> per i diagrammi, <code>../logo.jpg</code> per i loghi) per evitare errori di compilazione.
Versionamento	Disallineamento versione	Controllare che la versione indicata nel frontespizio coincida con l'ultima voce del registro delle modifiche.
Versionamento	Nome file senza versione	Il nome del documento deve contenere il numero di versione corrente (es. <code>Nome_doc_v1.0.0</code>).

Tabella 27: Checklist Documentazione (LaTeX)

Oggetto	Errore	Azione Correttiva
Sincronizzazione	Push senza Pull	Eseguire sempre <code>git pull</code> prima di <code>git push</code> per risolvere conflitti locali ed evitare merge complessi.
Pulizia	Codice commentato o morto	Rimuovere blocchi di codice commentato inutile o funzioni non utilizzate prima del push.

Tabella 28: Checklist Codice e Push (Git)

Oggetto	Errore	Azione Correttiva
Tracciamento	Requisiti per un caso d'uso assenti	Ad ogni caso d'uso deve corrispondere almeno un requisito.
Diagrammi	Diagrammi dei casi d'uso erranei	I diagrammi devono riportare la corretta numerazione e nomenclatura del caso d'uso.
Tracciamento	Caso d'uso senza requisiti	Ogni Caso d'Uso (UC) deve generare almeno un Requisito.
UML	Inconsistenza diagrammi	La numerazione e i nomi negli schemi UML devono corrispondere esattamente al testo descrittivo.
Attori	Attori non definiti	Verificare che tutti gli attori (es. Utente, Sistema AI) siano descritti nella sezione introduttiva degli UC.
Nomenclatura	Codice identificativo errato	Ogni requisito deve avere un codice univoco che segue la nomenclatura standard definita.

Tabella 29: Checklist Analisi Dei Requisiti

3.2 Analisi Dinamica_G

L'analisi dinamica del software *Second Brain* viene condotta attraverso l'esecuzione di test strutturati su diversi livelli di granularità, secondo il modello a V del ciclo di vita del software. L'obiettivo è verificare che il sistema soddisfi i requisiti funzionali e non funzionali, garantendo affidabilità, correttezza e usabilità del prodotto finale.

Priorità di Testing (per la fase RTB) Durante lo sviluppo del Proof of Concept (PoC), l'analisi dinamica si concentra sulla verifica della fattibilità tecnologica e sull'integrazione tra i componenti. Le priorità sono così definite:

- **Alta:** Test di Sistema (funzionalità AI core) e Test di Integrazione (API Auth + AI, Backend-Database).
- **Media:** Test di Unità (logiche isolate come `auth.py` o formattazione dei prompt).

L'obiettivo attuale non è raggiungere una *Code Coverage* esaustiva (che sarà centrale nella fase PB), ma validare che i rischi tecnologici siano stati mitigati.

3.2.1 Test di Unità_G

Obiettivo: Verificare il corretto funzionamento delle singole unità software in isolamento (funzioni, metodi, moduli), assicurando che producano l'output atteso per ogni input fornito.

Test ID	Funzione Testata	Descrizione	Input	Output Atteso
TU-1	<code>genera_hash()</code>	Verifica generazione hash PBKDF2 con salt casuale.	pwd: "test123"	Stringa formato <code>salt\$hash</code>
TU-2	<code>verifica_pwd()</code>	Verifica confronto con una password corretta.	pwd: "test123", hash valido	True
TU-3	<code>verifica_pwd()</code>	Verifica rifiuto con una password errata.	pwd: "wrong", hash valido	False
TU-4	<code>verifica_pwd()</code>	Gestione delle eccezioni per hash malformati o nulli.	hash malformato	False

Tabella 30: Test di Unità

3.2.2 Test di Integrazione_G

Obiettivo: Verificare che i diversi moduli del sistema (Frontend, Backend, Database e LLM esterni) comunichino correttamente tra loro attraverso le rispettive interfacce e API.

Test ID	Descrizione	Requisito
TI-1	Health Check Database: Verifica che il backend riesca a stabilire una connessione stabile con il database PostgreSQL interrogando l'endpoint dedicato (<code>/api/test-db-connection</code>) ed eseguendo una query di test.	ROP-F-22
TI-2	Health Check LLM: Verifica in fase di avvio dell'applicazione che le variabili d'ambiente (API Key e Base URL) siano correttamente caricate e che il client verso il provider esterno venga inizializzato con successo.	RO-V-13
TI-3	Verifica che la chiamata di registrazione dal backend salvi correttamente e persistentemente il record del nuovo utente all'interno del database PostgreSQL.	ROP-F-11
TI-4	Verifica che l'endpoint del backend formatti correttamente il prompt richiesto e lo inoltri con successo alle API del provider LLM, ricevendo una risposta valida.	RO-V-13
TI-5	Verifica che la funzione frontend <code>loginUtente()</code> componga correttamente il body JSON della richiesta HTTP POST e gestisca adeguatamente la risposta del backend (es. token di sessione o errore).	ROP-F-9

Tabella 31: Test di Integrazione

3.2.3 Test di Sistema_G

Obiettivo: Validare il comportamento del software nel suo complesso, simulando i percorsi d'uso (End-to-End) dell'utente finale tramite l'interfaccia grafica per garantire la copertura dei casi d'uso.

Test ID	Descrizione	Caso d'Uso (UC)
TS-1	Flusso Autenticazione: L'utente accede alla pagina di login, inserisce credenziali valide e preme "Accedi". Il sistema lo reindirizza all'editor, nasconde i bottoni di login/registrazione e l'area utente.	UC-5.0, UC-5.5
TS-2	Flusso AI Core: L'utente carica un file locale, ne seleziona una porzione, richiede un'operazione all'AI (es. "Riassumi"). Il sistema elabora la richiesta e inietta il risultato nel pannello "Storico Generazioni".	UC-2.*
TS-3	Flusso Editor e I/O: L'utente scrive testo formattato in Markdown nell'editor, verifica che la sintassi venga riconosciuta, ed esegue il salvataggio (download) del file in formato .md sul proprio dispositivo.	UC-1.0, UC-4.0

Tabella 32: Test di Sistema

4 Cruscotto di Valutazione della Qualità_G

In questa sezione vengono presentati gli indicatori di qualità relativi al processo produttivo, al fine di valutarne l'andamento durante le due fasi di progetto: RTB (Requirements and Technology Baseline) e PB (Product Baseline).

4.1 Qualità di Processo

4.1.1 Estimated at Completion (EAC)

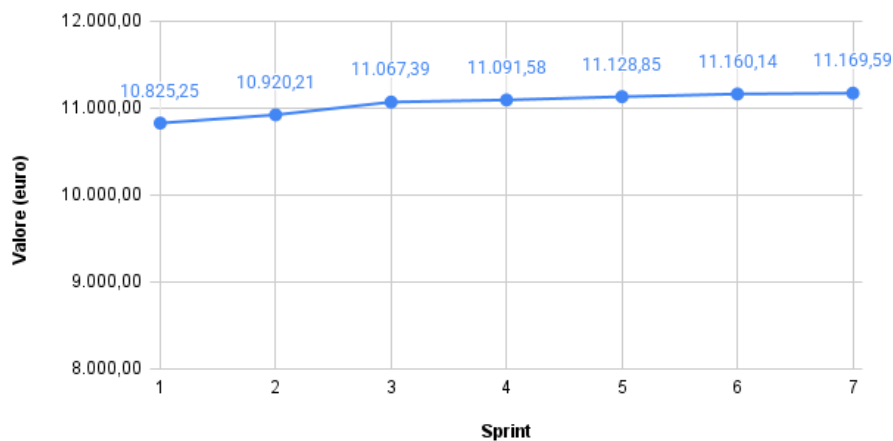


Figura 1: Andamento del valore di Estimated at Completion (EAC) nella fase RTB

RTB: L'analisi dell'Estimated at Completion (EAC) mostra un andamento che si è mantenuto costantemente al di sotto del preventivo totale iniziale (Budget at Completion, pari a 11.395 euro) per tutta la durata della fase. Nonostante alcune fluttuazioni iniziali e i rallentamenti temporali registrati negli Sprint 3 e 5, l'ottima efficienza sui costi mantenuta dal team (CPI sempre ≥ 1) ha permesso di stabilizzare la stima finale verso il basso. Il gruppo chiude quindi la fase RTB con una proiezione economica solida, confermando l'assenza di rischi legati allo sforamento del budget assegnato.

4.1.2 Planned Value (PV) e Earned Value (EV)

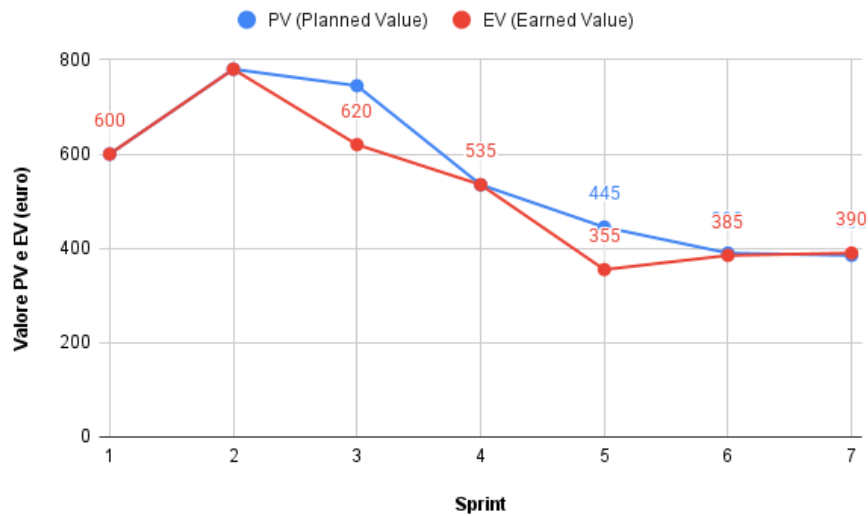


Figura 2: Andamento e confronto di PV e EV nella fase RTB

RTB: Dal confronto tra le curve di Planned Value (costo pianificato) ed Earned Value (valore prodotto), emerge una flessione dell'EV durante lo Sprint 3 e lo Sprint 5 a causa della ridotta disponibilità oraria durante il periodo natalizio e per gli esami invernali.

Tuttavia, analizzando anche l'Actual Cost (AC, non rappresentato in figura ma rendicontato nei consuntivi), emerge che i costi effettivi si sono mantenuti sempre allineati o inferiori all'EV: questo dimostra che le spese sono state costantemente sotto controllo e proporzionate al lavoro realmente svolto, senza alcuno spreco. Durante gli Sprint 6 e 7 il team ha incrementato la produttività, riportando l'EV ad allinearsi al PV e chiudendo la fase senza debiti temporali e con un risparmio effettivo ($AC < PV$).

4.1.3 Schedule Performance Index (SPI)

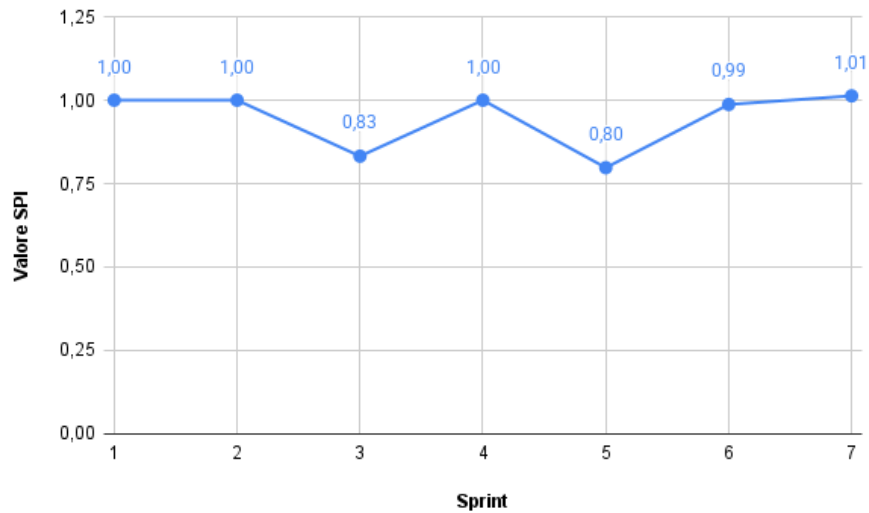


Figura 3: Andamento dello Schedule Performance Index (SPI) nella fase RTB

RTB: L'indice di efficienza della schedulazione (SPI) evidenzia visibilmente l'impatto della sessione d'esami, toccando un picco negativo di 0.83 nello Sprint 3 e di 0.80 nello Sprint 5. Questo scostamento si è tradotto in una Schedule Variance (SV) negativa, che ha toccato un picco di -125 € nello Sprint 3.

Tuttavia, la redistribuzione dei carichi di lavoro ha permesso di recuperare il gap già negli Sprint successivi (4 e 6) e di chiudere lo Sprint 7 in perfetto orario (SPI a 1.01) e con un saldo temporale tornato in positivo ($SV = +5$ €). L'andamento a "V" del grafico dimostra un'ottima capacità di reazione del team agli imprevisti temporali. L'obiettivo del team è rendere l'indice più stabile nella prossima fase di PB attraverso una migliore pianificazione delle attività.

4.1.4 Cost Performance Index (CPI)

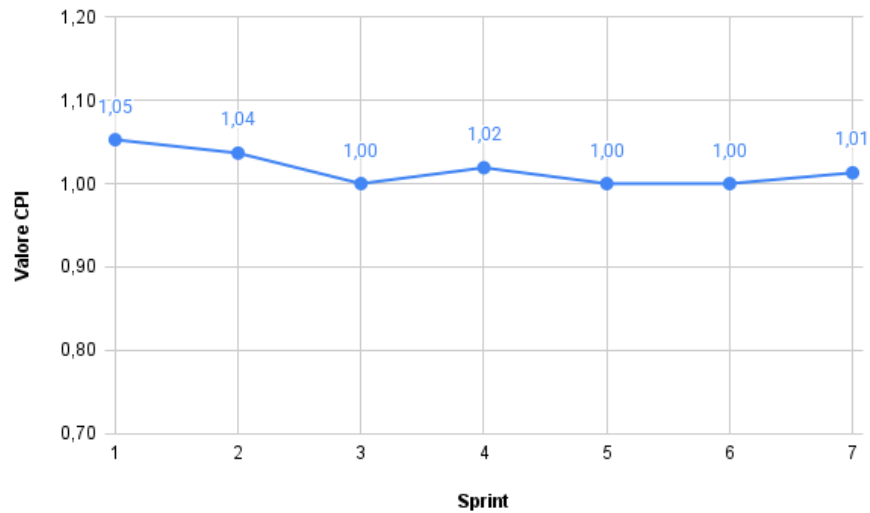


Figura 4: Andamento del Cost Performance Index (CPI) nella fase RTB

RTB: L'indice di efficienza dei costi (CPI) mostra un andamento eccellente, mantenendosi costantemente attorno al valore 1.0 o superiore. Ciò significa che per ogni euro speso dal progetto, il valore prodotto è stato pari o superiore a un euro. Nonostante il ritardo temporale (SPI $\leq$ 1) negli Sprint 3 e 5, il gruppo è stato molto efficiente economicamente: le ore lavorate sono state produttive e non ci sono stati sprechi di risorse o rilavorazioni costose che avrebbero abbassato il CPI.

4.1.5 Cost Variance (CV)

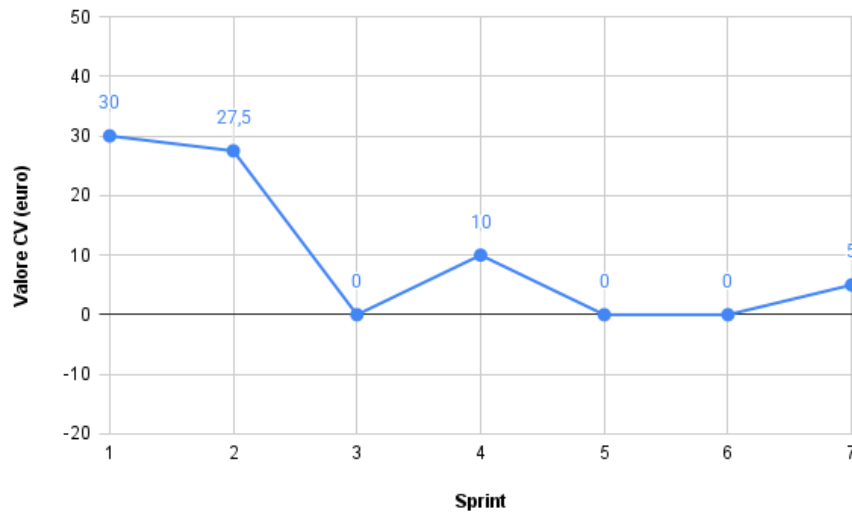


Figura 5: Andamento della Cost Variance (CV) nella fase RTB

RTB: La varianza dei costi (CV), calcolata come differenza tra Earned Value e Actual Cost ($EV - AC$), si è mantenuta positiva o prossima allo zero per tutta la durata della fase. Questo indicatore conferma che il gruppo non ha mai speso più del valore generato. Al termine della RTB, il risparmio cumulato è pari a 72,50 €, garantendo al progetto un prezioso margine di sicurezza economico da investire nella successiva fase di Product Baseline.

4.1.6 Requirements Stability Index (RSI)

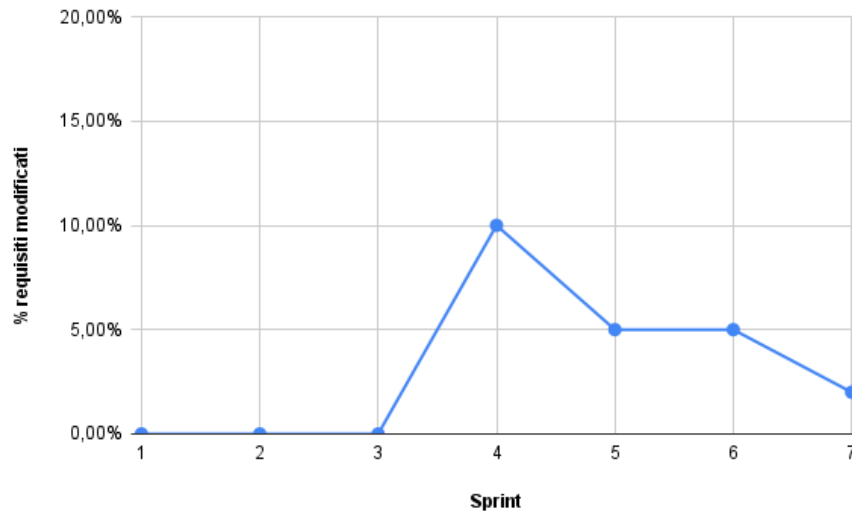


Figura 6: Andamento del Requirements Stability Index (RSI) nella fase RTB

RTB: L'indice di stabilità dei requisiti ha subito alcune variazioni in particolare durante lo Sprint 4 e 5. A seguito del colloquio di verifica con il prof. Cardin, il gruppo ha dovuto apportare correzioni e raffinamenti ai casi d'uso e ai requisiti precedentemente individuati. Negli ultimi sprint (Sprint 6), con il consolidamento dell'architettura per il PoC, l'indice si è stabilizzato, indicando che il perimetro del progetto è ora ben definito.

4.1.7 Rischi Non Calcolati (RNC)

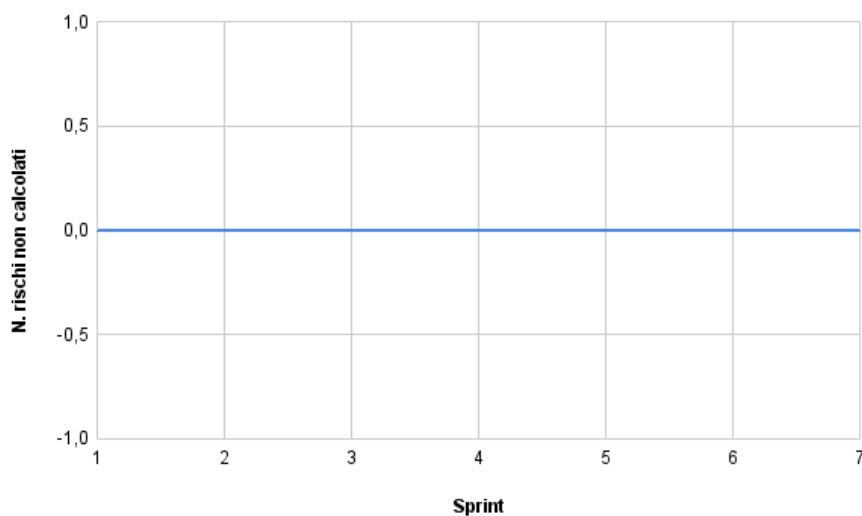


Figura 7: Numero di rischi non calcolati (RNC) nella fase RTB

RTB: Durante la fase RTB, il numero di rischi imprevisti è stato pari a zero (o molto basso). Le principali criticità riscontrate, come la ridotta disponibilità dovuta agli esami (RO-1) e le difficoltà di apprendimento delle tecnologie LLM (RT-1), erano state correttamente previste e mitigate. Il gruppo può ritenersi soddisfatto della capacità di previsione e gestione dei rischi.

4.1.8 Metriche Soddisfatte (MS)

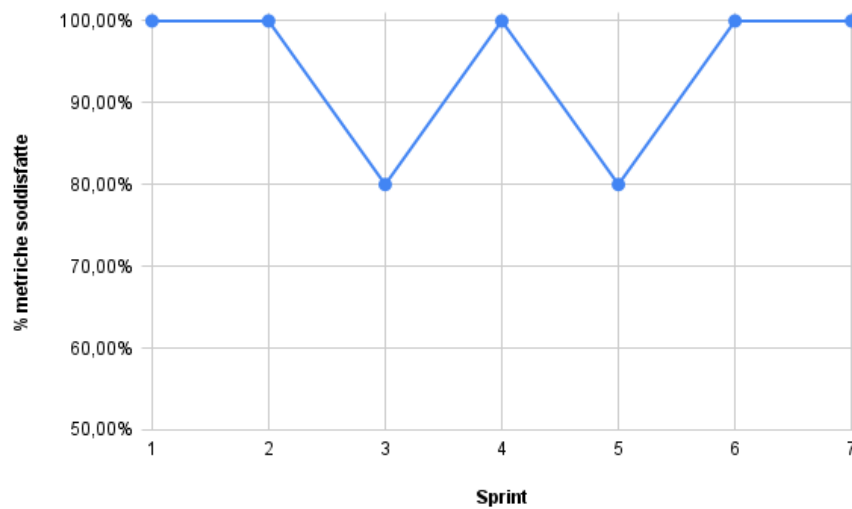


Figura 8: Percentuale delle metriche soddisfatte (MS) nella fase RTB

RTB: La percentuale complessiva di metriche soddisfatte ha registrato due flessioni (attorno all'80%) in corrispondenza degli Sprint 3 e 5, principalmente a causa degli sforamenti temporali ($SPI \geq 0.90$). Tuttavia, la tenuta impeccabile delle metriche economiche (CPI, CV sempre ottimali) e il recupero finale hanno permesso di chiudere la fase con il 100% degli indicatori in stato accettabile o ottimale. Il processo adottato è pertanto ritenuto maturo e validato.